

Array

Gli array sono delle variabili particolari, che consentono di memorizzare più valori in una singola variabile anziché dichiarare più variabili. In C, gli array devono essere omogenei, ovvero devono contenere dati dello stesso tipo.

Possiamo vedere gli array come una "cassettiera", suddivisa in celle numerate; ognuna di queste celle, quindi, occupa una ben specifica posizione. La prima cella dell'array è sempre in posizione 0, l'ultima cella ha posizione n-1, dove n è la dimensione dell'array.

Dichiarazione

Per dichiarare un array, come per qualsiasi altro tipo di variabile, è necessario indicarne il tipo ed un nome, seguito da '['].

Esempi:

```
int numeri[] = {1, -4, 55, 76};  
float medie[5];
```

Nel primo esempio abbiamo dichiarato un array di interi, contenente quattro valori, mentre nel secondo esempio abbiamo dichiarato un array di float di dimensione 5.

Quando in fase di dichiarazione non riempiamo l'array con degli elementi, è necessario specificare la dimensione dell'array.

N.B.: in C gli array sono statici, ovvero la loro dimensione non può cambiare durante l'esecuzione del programma. Pertanto, non è possibile aggiungere o eliminare elementi ad un array.

Accedere ad un elemento

Come detto, ogni elemento dell'array è memorizzato in una cella, occupando una ben determinata posizione; quindi, per poter accedere ad un qualsiasi elemento, occorre indicare l'indice dove trovarlo. Per fare questo, occorre indicare il nome dell'array, specificando tra '[']' l'indice dell'elemento a cui vogliamo accedere.

Chiaramente l'accesso ad un elemento può essere fatto sia in lettura che in scrittura.

Esempio:

```
int numeri[] = {1, -4, 55, 76};  
  
printf("%d\n", numeri[1]);  
  
numeri[1] = 33;  
  
printf("%d\n", numeri[1]);
```

In questo esempio, la prima istruzione `printf()` stamperà il valore -4, mentre la seconda istruzione stamperà 33, perché con l'istruzione `numeri[1] = 33;` andiamo a cambiare il valore in posizione 1 del nostro array.

Per comodità, si utilizzano i cicli per scorrere un array; in particolare, i cicli `for`.

Esempio:

```
int numeri[] = {1, -4, 55, 76}, i;

for (i = 0; i < 4; i++){
    printf("%d\n", numeri[i]);
}
```

In questo modo, la variabile `i` del ciclo `for` funziona come indice dell'array, scorrendo dall'inizio alla fine.

Dimensione e lunghezza di un array

Possiamo conoscere la dimensione di un array utilizzando la funzione `sizeof()`. Tuttavia, occorre tenere ben presente che `sizeof()` non restituisce il numero di elementi contenuti nell'array, ma bensì la sua dimensione in byte.

Pertanto, in funzione del tipo di dato, la dimensione di un array può cambiare.

La sintassi è:

```
sizeof(nomeArray);
```

Esempio:

```
int numeri[] = {1, 44, -5, 100, 8};
printf("%d\n", sizeof(numeri));
```

In questo esempio il valore restituito da `sizeof(numeri)` è 20, indicante il numero di byte che occupa l'array in memoria (infatti ricordiamo che un intero occupa tendenzialmente 4 byte in memoria)

Conoscere la dimensione in byte di un array ci consente chiaramente di risalire alla sua lunghezza (il numero di elementi): infatti, è sufficiente dividere la dimensione dell'intero array per la dimensione di uno dei suoi elementi (per esempio il primo).

Esempio:

```
int numeri[] = {1, 44, -5, 100, 8};

int lunghezza = sizeof(numeri) / sizeof(numeri[0]);

printf("%d\n", lunghezza);
```

In questo caso la stampa comunicherà 5, che è la lunghezza del nostro array.

Ordinamento di un array

Quando usiamo array numerici, può capitare di volere i numeri ordinati in maniera crescente o decrescente; tuttavia, non sempre questa condizione è verificata in partenza (pensiamo ad esempio ad array inizializzati con valori casuali).

Esistono diversi algoritmi di ordinamento di un array, noi ne vedremo un paio molto semplici. Prima di procedere, però, dobbiamo sapere come poter scambiare il valore di due celle dell'array (in generale di due variabili).

Per poter fare uno scambio di valori tra due variabili non possiamo scrivere brutalmente:

```
int a = 4, b = 5;  
a = b;  
b = a;
```

Infatti, una volta che la variabile *a* assume il valore della variabile *b*, il suo precedente valore viene perso definitivamente.

Pertanto, occorre prevedere una terza variabile, una variabile di supporto temporanea, nella quale salvare il valore di una delle due variabili e procedere poi allo scambio.

Esempio:

```
int a = 4, b = 5, tmp;  
tmp = a;  
a = b;  
b = tmp;
```

Così facendo, quando ad *a* assegniamo il valore di *b*, il valore salvato in *a* non viene perso, in quanto è stato memorizzato in *tmp*.

Lo stesso ragionamento si applica alle celle di un array.

Esempio:

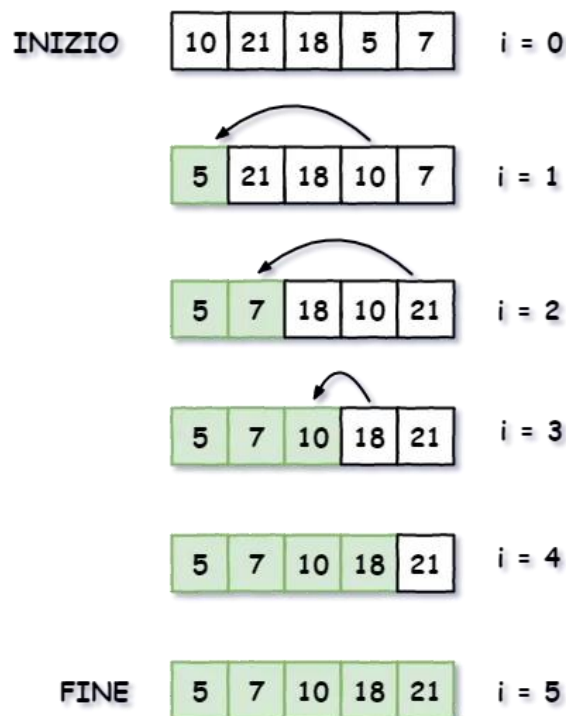
```
int numeri[] = {1, 44, -5, 100, 8}, tmp;  
  
tmp = numeri[0];  
numeri[0] = numeri[1];  
numeri[1] = tmp;
```

Selection Sort

Il primo algoritmo di ordinamento che vediamo è il Selection Sort, un algoritmo molto molto semplice.

Partendo dall'inizio dell'array ($i = 0$):

1. scorre l'array in cerca dell'elemento più piccolo
2. scambia l'elemento più piccolo con l'elemento in posizione i
3. incrementa il valore di i
4. ripete i passi precedenti fino a terminare l'array.



È facile capire il nome dell'algoritmo: ad ogni iterazione viene selezionato il valore più piccolo che dovrà essere spostato in posizione i .

Vantaggi:

- algoritmo molto semplice da implementare
- il numero massimo di scambi effettuati è pari alla dimensione dell'array.

Svantaggi:

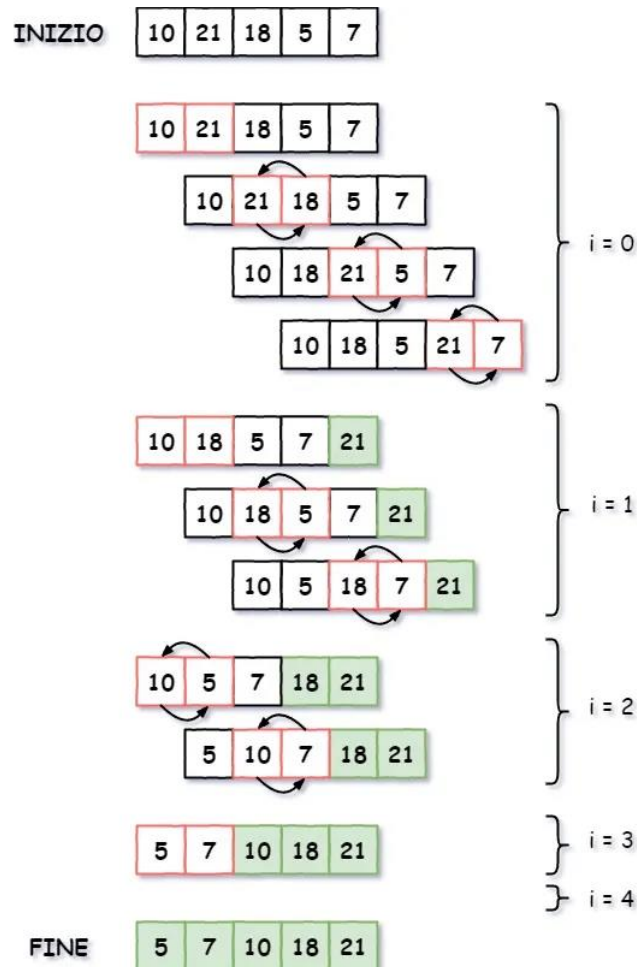
- al crescere del numero di elementi dell'array diventa più inefficiente
- la velocità dell'algoritmo non dipende da quanto questo è ordinato, in quanto ad ogni iterazione viene comunque cercato in tutto l'array il valore minimo; quindi per ordinare un array con solo una coppia di elementi fuori posto, il tempo impiegato sarà uguale (o poco inferiore) nel caso di un array completamente disordinato.

Bubble Sort

Anche il Bubble Sort è un algoritmo di ordinamento molto semplice e deve il suo nome al modo con cui ordina gli elementi, spostandoli "verso l'alto" come se fossero delle bollicine in una bevanda gassata.

Partendo dall'inizio dell'array:

1. prende i primi due elementi e se il primo è maggiore del secondo li scambia di posizione
2. prosegue scambiando gli elementi fino alla fine dell'array
3. continua così per tutte le iterazioni



Vantaggi:

- molto semplice da implementare

Svantaggi:

molto inefficiente: la velocità diminuisce drasticamente all'aumentare della dimensione dell'array